

Decision Trees

CS21206: Foundations of AI and ML

Abir Das & Ayan Chaudhury

IIT Kharagpur

Mar 13 and 19, 2026

Agenda

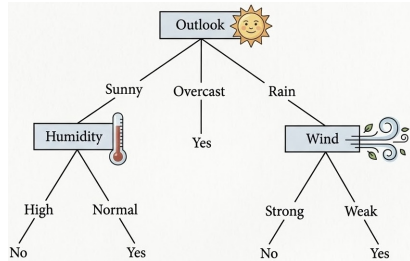
- § Making decision in decision trees
- § Understand how to build decision trees using different algorithms for it

Resources

- § Artificial Intelligence, A Modern Approach by Stuart J. Russell and Peter Norvig
- § Machine Learning by Tom M. Mitchel.

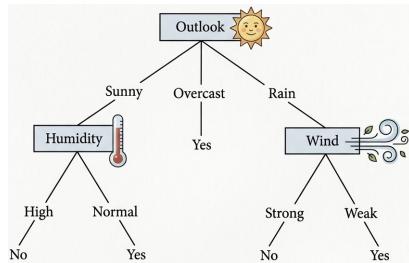
Decision Trees

- § A **Decision Tree** is one of the simplest and yet most successful ML algorithms.
- § A decision tree represents a function that takes as input a vector of attribute values and returns a “decision” — a single output value.
- § We will see that, decision trees can be thought of as sets of if-then-else rules for making decisions.
- § For now we will consider problems with inputs having discrete values and the output having exactly two possible values.



Decision Tree to decide on Playing Tennis

- § A decision tree reaches its decision by performing a sequence of tests.
- § Each internal node corresponds to a test of one of the input attributes.
- § This figure represents a typical **learned** decision tree.
- § It classifies Saturday mornings according to whether they are suitable for playing tennis.

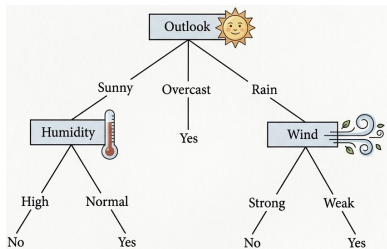


§ What decision is given for the instance - (Outlook=Sunny, Temperature=Hot, Humidity=High, Wind=Strong)?



§ What decision is given for the instance - (Outlook=Sunny, Temperature=Hot, Humidity=High, Wind=Strong)?

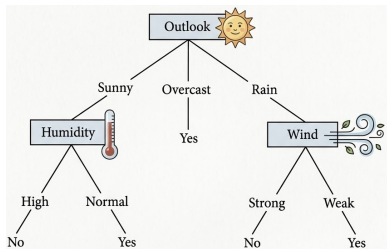
§ It would be sorted down to the left-most branch and would therefore predict `PlayingTennis = No`.



§ What decision is given for the instance - (Outlook=Sunny, Temperature=Hot, Humidity=High, Wind=Strong)?

§ It would be sorted down to the left-most branch and would therefore predict `PlayingTennis = No`.

§ **Wait!** Does it not consider the Temperature of the morning or how is the Wind?



§ What decision is given for the instance - (Outlook=Sunny, Temperature=Hot, Humidity=High, Wind=Strong)?

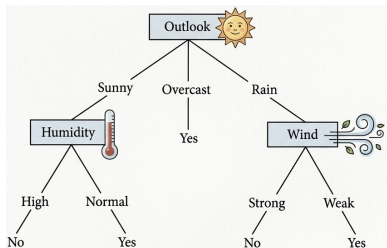
§ It would be sorted down to the left-most branch and would therefore predict `PlayingTennis = No`.

§ **Wait!** Does it not consider the Temperature of the morning or how is the Wind?

- § It doesn't have to use every available attribute to reach a decision.

§ A learning algorithm determines the most ‘informative’ attributes from a set of training data.

§ If the Outlook is Sunny, the algorithm has 'decided' that Humidity is the only other thing it needs to know.



Example Dataset: Waiting for a Restaurant Table

- § A training set of 12 examples. Positive examples have WillWait = Yes and negative examples have WillWait = No
- § Attributes include: Alternative, Bar, Friday, Patrons etc.

Ex	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	Yes
X_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	No
X_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

Learning a Decision Tree from Data

- § We want a tree that is consistent with the examples and is as small as possible. Note: A hypothesis is *consistent* when the hypothesis fits/agrees with all the training data.
- § It is an intractable problem to efficiently search such a tree from an exponential number of possible trees.
- § With some simple heuristics, we can find a good approximate solution: a small (but not smallest) consistent tree.

- § We want a tree that is consistent with the examples and is as small as possible. Note: A hypothesis is *consistent* when the hypothesis fits/agrees with all the training data.
- § It is an intractable problem to efficiently search such a tree from an exponential number of possible trees.
- § With some simple heuristics, we can find a good approximate solution: a small (but not smallest) consistent tree.
- § We can view this problem as playing a 20 questions game, where every question we ask:
 - ▶ should help us in narrowing down the value of WillWait.
 - ▶ depends on the previous questions that we may have already asked.

Learning a Decision Tree from Data

- § We want a tree that is consistent with the examples and is as small as possible. Note: A hypothesis is *consistent* when the hypothesis fits/agrees with all the training data.
- § It is an intractable problem to efficiently search such a tree from an exponential number of possible trees.
- § With some simple heuristics, we can find a good approximate solution: a small (but not smallest) consistent tree.
- § We can view this problem as playing a 20 questions game, where every question we ask:
 - ▶ should help us in narrowing down the value of WillWait.
 - ▶ depends on the previous questions that we may have already asked.
- § What happens when we ask a question?
 - ▶ Depending upon the possible answers data is split into multiple subsets.
 - ▶ If a subset has the same value for the target concept (e.g., WillWait) then we have our answer.
 - ▶ If the subset has different values for the target concept, then we need to ask more questions.

- § Which question should we ask *i.e.*, which attribute should we pick?
- § Pick the one that makes the most difference to the classification.
- § That way, we hope the correct answer to come with a small number of tests.

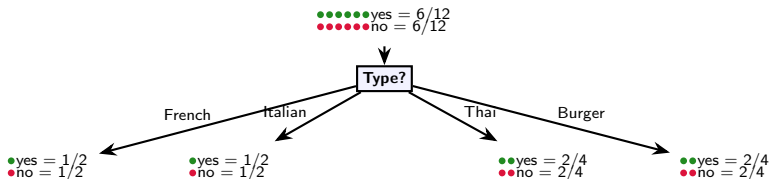
- § Which question should we ask *i.e.*, which attribute should we pick?
- § Pick the one that makes the most difference to the classification.
- § That way, we hope the correct answer to come with a small number of tests.
- § For the data shown in the previous table, let's see what happens when we ask two different questions.
 - ▶ patron: How many persons are in the restaurant currently (Some, Full, None)
 - ▶ type: What kind of restaurant is this (French, Italian, Thai, Burger)

How about Type?

Ex	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	Yes
X_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	No
X_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10	Yes
X_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	Yes
X_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	Yes
X_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	No
X_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	Yes
X_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	No
X_{11}	No	No	No	No	None	\$	No	No	Thai	0-10	No
X_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	Yes

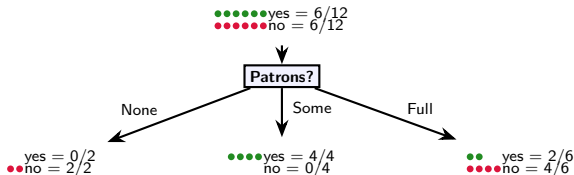
How about Type?

Ex	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0–10	Yes
X_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30–60	No
X_3	No	Yes	No	No	Some	\$	No	No	Burger	0–10	Yes
X_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10–30	Yes
X_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0–10	Yes
X_7	No	Yes	No	No	None	\$	Yes	No	Burger	0–10	No
X_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0–10	Yes
X_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10–30	No
X_{11}	No	No	No	No	None	\$	No	No	Thai	0–10	No
X_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30–60	Yes

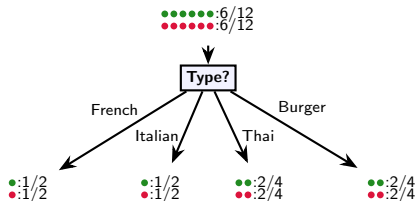


How about Patrons?

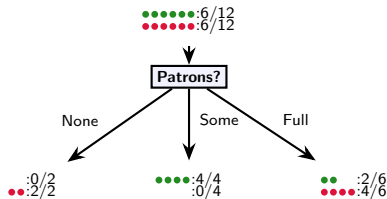
Ex	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
X_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0–10	Yes
X_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30–60	No
X_3	No	Yes	No	No	Some	\$	No	No	Burger	0–10	Yes
X_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10–30	Yes
X_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	No
X_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0–10	Yes
X_7	No	Yes	No	No	None	\$	Yes	No	Burger	0–10	No
X_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0–10	Yes
X_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	No
X_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10–30	No
X_{11}	No	No	No	No	None	\$	No	No	Thai	0–10	No
X_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30–60	Yes



Comparing Type and Patrons



- § Starts with same number of positive and negative examples and ends with four possible outcomes, each has same number of positive and negative examples.



§ Starts with same number of positive and negative examples. If the value is None or Some, we can answer definitively. If the value is Full, we are left with a mixed set.

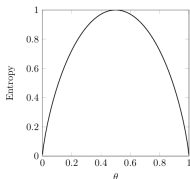
§ The idea is to pick the attribute that goes as far as possible toward providing an exact classification of the examples.

- § The idea is to pick the attribute that goes as far as possible toward providing an exact classification of the examples.
- § A perfect attribute divides the examples into sets, each of which are all positive or all negative.
- § The Patrons attribute is not perfect, but it is fairly good.
- § Type is an almost useless attribute - leaves the example sets with roughly the same proportion of positive and negative examples as the original set.

13 / 37

Entropy (Recap)

- § The entropy of a discrete random variable with PMF P over K states is defined by, $H(p) = -\mathbb{E}_P[\log P(x)] = -\sum_{i=1}^K P(x_i) \log P(x_i)$
- § The entropy of a probability distribution can be interpreted as a measure of uncertainty or lack of predictability
- § For a Bernoulli random variable $p(x; \theta) = \theta^x (1 - \theta)^{1-x}$, the entropy is: $H(p) = -(\theta \log \theta + (1 - \theta) \log(1 - \theta))$



- § Entropy of a Bernoulli random variable as a function of θ and with log base 2.

§ Maximum entropy is when the distribution is uniform, $\theta = 0.5$.

§ Minimum entropy is when one of the probabilities is 1, *i.e.*, there is no uncertainty.

§ An uniform distribution has the highest entropy while a distribution with all probability mass concentrated at a single value has the minimum entropy.

Intuition: The Quest for Certainty

- § We are starting with a mixed set of examples (**6/12 Yes, 6/12 No**).
- § Ideally, after trying one attribute (*Patrons, Type, Bar, etc.*), we would like to reach a pure state:
 - ▶ **All Yes** ($H = 0$) OR **All No** ($H = 0$)

Intuition: The Quest for Certainty

- § We are starting with a mixed set of examples (**6/12 Yes, 6/12 No**).
- § Ideally, after trying one attribute (*Patrons, Type, Bar, etc.*), we would like to reach a pure state:
 - ▶ **All Yes** ($H = 0$) OR **All No** ($H = 0$)

In terms of Entropy:

- § **Starting State:** $H([0.5, 0.5]) = 1$. Any test input has an equal chance of being classified as Yes or No.
- § **Ideal State:** Either $H([1, 0])$ or $H([0, 1])$, both of which are 0.
- § **Result:** After testing that attribute, the input should have a 100% chance of belonging to either Yes or No.

Intuition: The Quest for Certainty

- § We are starting with a mixed set of examples (**6/12 Yes, 6/12 No**).
- § Ideally, after trying one attribute (*Patrons, Type, Bar, etc.*), we would like to reach a pure state:
 - ▶ **All Yes** ($H = 0$) OR **All No** ($H = 0$)

In terms of Entropy:

- § **Starting State:** $H([0.5, 0.5]) = 1$. Any test input has an equal chance of being classified as Yes or No.
- § **Ideal State:** Either $H([1, 0])$ or $H([0, 1])$, both of which are 0.
- § **Result:** After testing that attribute, the input should have a 100% chance of belonging to either Yes or No.
- § **Practically:**
 - ▶ We may not always start at a node with the highest entropy (1.0).
 - ▶ But until and unless all examples at a node belong to the same class, it will have some non-zero entropy.
 - ▶ Goal is to reach the **lowest entropy possible** after choosing an attribute.

How to pick nodes?

§ Suppose, at a node in a decision tree we have p positive examples and n negative examples. Then the entropy at this node is:

$$H\left(\left[\frac{p}{p+n}, \frac{n}{p+n}\right]\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

How to pick nodes?

§ Suppose, at a node in a decision tree we have p positive examples and n negative examples. Then the entropy at this node is:

$$H\left(\left[\frac{p}{p+n}, \frac{n}{p+n}\right]\right) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

§ Suppose the attribute A , with K distinct values, divides the training set E into subsets $E_1, \dots, E_K$, with p_i and n_i be the positive and negative examples in subset E_i

§ The entropy at the i^{th} branch is then, $H\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$

§ The **Expected Entropy** after trying attribute A having K branches:

$$EH(A) = \sum_{i=1}^K \frac{p_i + n_i}{p + n} H\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

How to pick nodes?

- § Before trying A, the entropy was $H\left(\left[\frac{p}{p+n}, \frac{n}{p+n}\right]\right)$. We want to reach entropy = 0 (ultimately).
- § After trying A, we reached $EH(A)$.
- § So, we are closer to our goal (entropy = 0) by

$$I(A) = H\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - EH(A)$$

How to pick nodes?

- § Before trying A , the entropy was $H\left(\left[\frac{p}{p+n}, \frac{n}{p+n}\right]\right)$. We want to reach entropy = 0 (ultimately).
- § After trying A , we reached $EH(A)$.
- § So, we are closer to our goal (entropy = 0) by

$$I(A) = H\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - EH(A)$$

- § This quantity $I(A)$ is called the **Information gain** (I) or reduction in entropy/uncertainty for this attribute.
- § **Selection Criterion:** Choose the attribute with the **largest** I , i.e., the attribute that reduces the uncertainty the most.

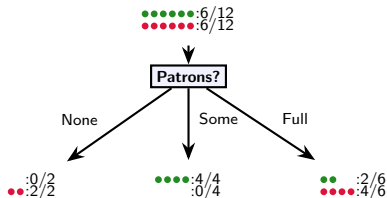
Example

- § Initially, $p = n = 6$. So the initial entropy is: $H(6/12, 6/12) = 1$ bit
- § Consider the attributes Patron and Type (and others too):

§ Initially, $p = n = 6$. So the initial entropy is: $H(6/12, 6/12) = 1$ bit

§ Consider the attributes Patron and Type (and others too):

► $I(Patron) = 1 - \left[\frac{2}{12}H(0, 1) + \frac{4}{12}H(1, 0) + \frac{6}{12}H\left(\frac{2}{6}, \frac{4}{6}\right) \right] = 1 - \left[0 + 0 + \frac{1}{2}(0.918) \right] = \mathbf{0.541 \text{ bits}}$



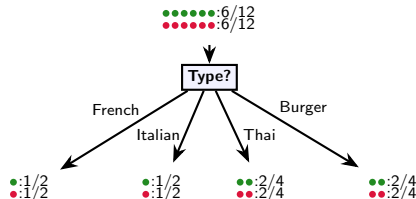
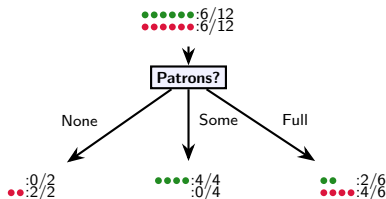
Example

§ Initially, $p = n = 6$. So the initial entropy is: $H(6/12, 6/12) = 1$ bit

§ Consider the attributes Patron and Type (and others too):

▶ $I(Patron) = 1 - \left[\frac{2}{12}H(0, 1) + \frac{4}{12}H(1, 0) + \frac{6}{12}H\left(\frac{2}{6}, \frac{4}{6}\right) \right] =$
 $1 - \left[0 + 0 + \frac{1}{2}(0.918) \right] = \mathbf{0.541}$ bits

▶ $I(Type) =$
 $1 - \left[\frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right) \right] =$
 $1 - \left[\frac{1}{6}(1) + \frac{1}{6}(1) + \frac{1}{3}(1) + \frac{1}{3}(1) \right] = 1 - 1 = \mathbf{0}$ bits



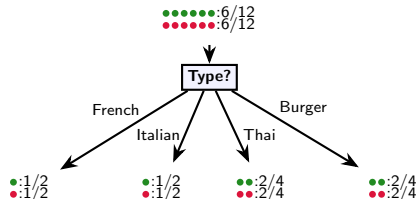
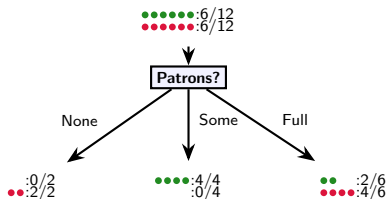
Example

§ Initially, $p = n = 6$. So the initial entropy is: $H(6/12, 6/12) = 1$ bit

§ Consider the attributes Patron and Type (and others too):

▶ $I(\text{Patron}) = 1 - \left[\frac{2}{12}H(0, 1) + \frac{4}{12}H(1, 0) + \frac{6}{12}H\left(\frac{2}{6}, \frac{4}{6}\right) \right] =$
 $1 - \left[0 + 0 + \frac{1}{2}(0.918) \right] = \mathbf{0.541}$ bits

▶ $I(\text{Type}) =$
 $1 - \left[\frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12}H\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{4}{12}H\left(\frac{2}{4}, \frac{2}{4}\right) \right] =$
 $1 - \left[\frac{1}{6}(1) + \frac{1}{6}(1) + \frac{1}{3}(1) + \frac{1}{3}(1) \right] = 1 - 1 = \mathbf{0}$ bits



§ This confirms our intuition that Patron is a better attribute than Type

The Final Decision Tree

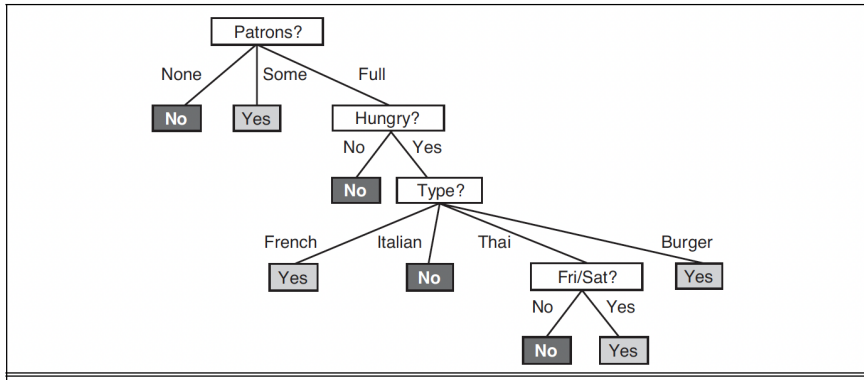


Figure credit: AIMA: Russell, Norvig

When to Stop? Stopping Criteria

The tree-building process is recursive. We consider four cases at any node:

- 1 **Pure Subset:** If the remaining examples are all positive (or all negative), then we are done.
- 2 **Mixed Examples:** If there are both positive and negative examples, **choose the best attribute** (highest Information Gain) to split them.
- 3 **No Examples Left:** If no examples reach this branch, return a default value based on the **plurality/majority classification** of the parent.
- 4 **No Attributes Left:** If we have mixed examples but no attributes remaining, it means examples have identical descriptions but different labels. Return the **plurality/majority classification** of the parent.

Gini Index: An Alternative Impurity Measure

- § Besides Entropy, the **Gini Index** is another measure of impurity.
- § For a node q with k classes, where p_i is the proportion of instances belonging to class i , the Gini Index is defined as:

$$Gini(q) = \sum_{i=1}^k p_i(1 - p_i) = 1 - \sum_{i=1}^k p_i^2$$

§ Properties:

- ▶ Reaches **minimum (0)** when all instances belong to one class (pure).
- ▶ Reaches **maximum $(1 - 1/k)$** when instances are equally distributed.

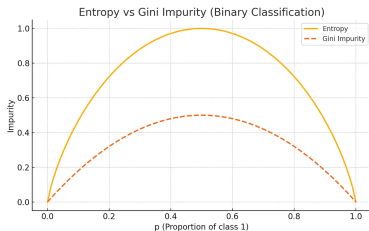


Fig. credit: [Crack Machine Learning with Decision Trees](#)

Gini Index: An Alternative Impurity Measure

§ Similar to Information Gain, we calculate the **Gini Gain** for a split on attribute A :

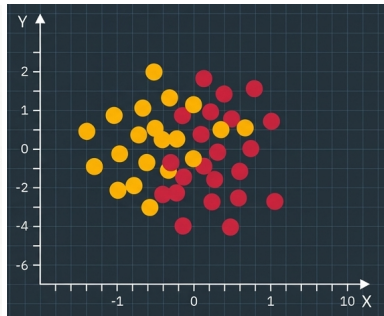
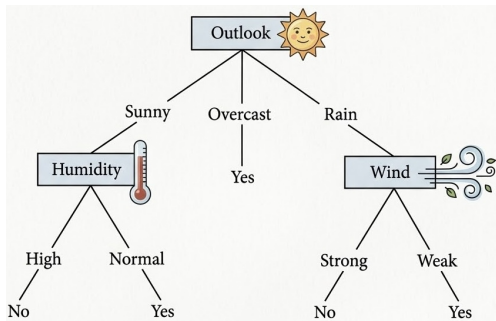
$$Gain_{Gini}(q, A) = Gini(q) - \sum_{i=1}^{|V|} \frac{N_i}{N_q} Gini(i)$$

(where N_i is the number of instances in the i^{th} child node)

§ **Selection:** We choose the attribute that **maximizes** this gain.

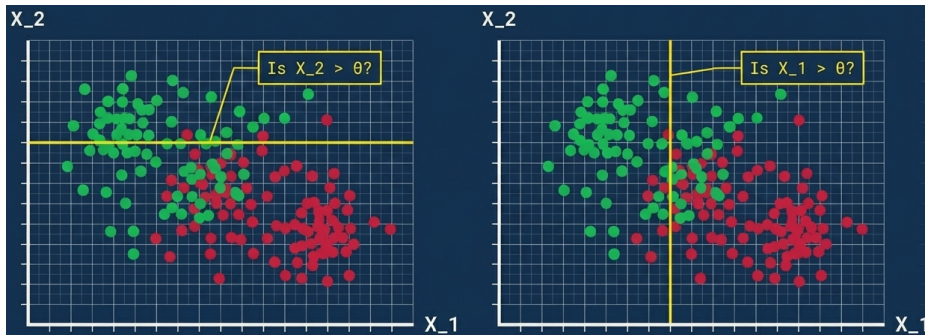
Handling Continuous Attributes

How does an algorithm built on 'if/then' logic handle a continuous spectrum of attribute values!



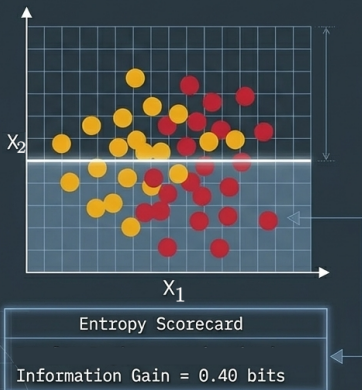
Handling Continuous Attributes

Orthogonal thresholds split the continuous spectrum into binary or even multiple regions.

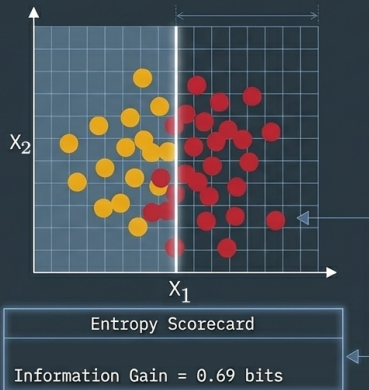


Finding the Optimal Cut

Candidate A: The Sub-optimal Cut

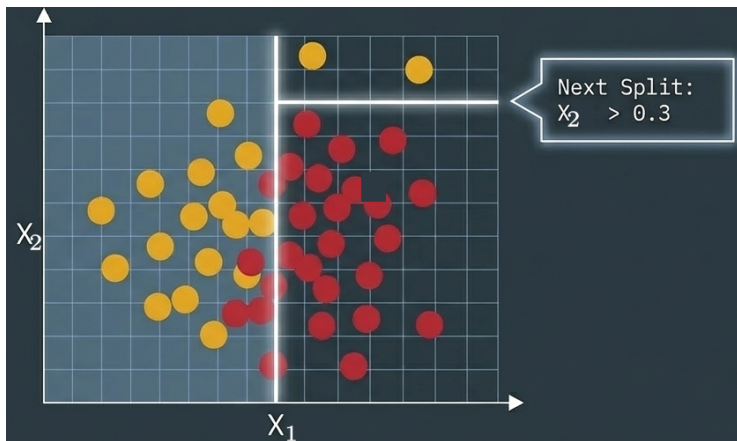


Candidate B: The Optimal Cut



Select Candidate B as the information gain is higher for candidate B.

Recursively Building the Tree



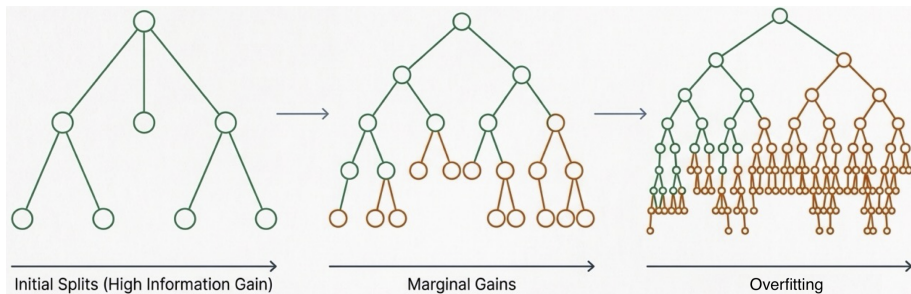
One threshold creates two spaces. New thresholds are recursively selected in each sub-region progressively drawing orthogonal boundaries.

The Problem: Growing the Tree too Large

- § If we continue adding nodes based on our stopping criteria, the decision tree will become larger and larger.
- § The error on the **training examples will reduce continuously**.

The Problem: Growing the Tree too Large

- § If we continue adding nodes based on our stopping criteria, the decision tree will become larger and larger.
- § The error on the **training examples will reduce continuously**.
- § However, often, as you continue to grow the tree, the error on unseen data will start to increase.
- § This is nothing but **Overfitting**.



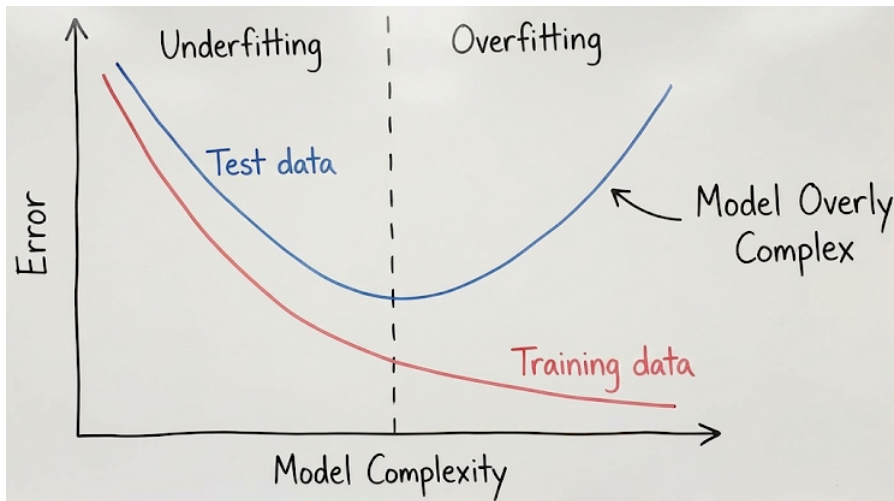
Formal Definition of Overfitting

We say that a hypothesis h is said to **overfit** the training data if there exists some alternative hypothesis h' such that:

- 1 h has a **smaller error** than h' on the **training data**, but
- 2 h has a **larger error** than h' on the **test data**.

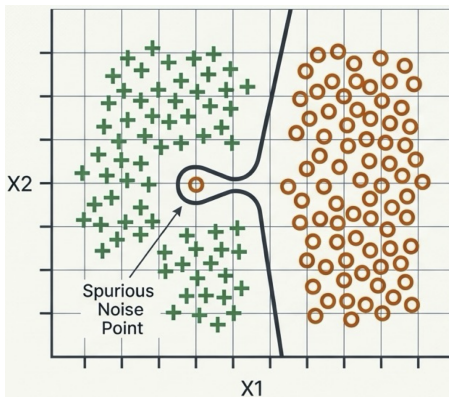
Example: A large, complex decision tree might perfectly classify the training data (zero error) but perform poorly on test data, while a smaller, simpler tree might have slight training errors but better test performance.

Training Error vs Test Error



Primary Reasons of Overfitting in Decision Trees

- § **Noise in the Training Data:** We want good generalization, but training data may contain noise (spurious points).
- § As the tree becomes complex, it tries to create specific decision boundaries to include these noise points, causing it to fail on the true population.



- § With perfect training accuracy, it may have very unusual boundaries due to noise. It does not generalize well.
- § It has high variance - with new data, the tree will change a lot.



- § **The Staircase effect:** Decision trees have a structural constraint that stems from its axis-aligned splits.
- § Consider a simple case where the optimal boundary is a diagonal line in a two-dimensional feature space. Because decision trees can only make splits parallel to the feature axes, they must approximate this by a series of rectangular regions, creating a stair-like pattern.

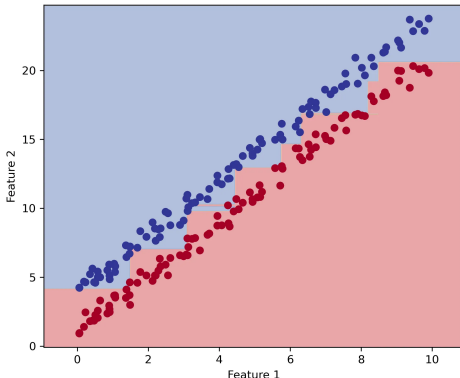


Fig. credit: *Climbing trees 1: what are decision trees?*

Dealing with Overfitting in Decision Trees

Crucial Takeaway: Training error is **not a good predictor** for test/true error. You must use a separate, unseen sample called the **validation set** to get a better estimate of the true error.

There are two standard types of methods to prevent overfitting in decision trees:

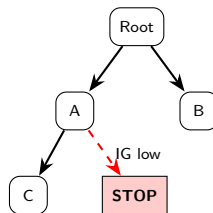
- 1 **Pre-pruning:** Pruning the tree while growing it by stopping early.
- 2 **Post-pruning:** Growing a full tree first, and then pruning sub-trees from the bottom up.

In **Pre-pruning**, we evaluate the split *before* adding a node. If it doesn't look worthwhile, we stop growing that branch.

§ **Node Size:** Number of instances is too small (e.g., < 5).

- § **Max Depth:** Tree has reached a pre-defined maximum depth.

§ **IG Threshold:** The Information Gain is below a tiny value ϵ .

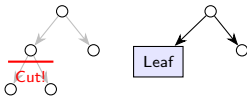


Early stopping prevents capturing noise.

Strategy: "Grow first, cut later."

- 1 Build a **full tree** on the training data.
- 2 Use a separate **Validation Set** to identify nodes that increase error.
- 3 For each node: Try replacing its entire subtree with a single leaf.
- 4 If validation accuracy improves (or stays same) → **Prune it**.

Benefit: This avoids the "horizon effect" where a bad split is needed to reach a great split later.



A specific and popular post-pruning approach:

- 1 Split the available data into a **Grow Set** and a **Validation Set**.
- 2 Build a complete tree using only the Grow Set (zero error if possible).
- 3 For each non-leaf node, temporarily prune the tree below that node and replace it with a **majority vote** label.
- 4 Test the accuracy of this pruned tree on the Validation Set.
- 5 **Permanently prune** the node that results in the greatest increase in validation accuracy.
- 6 Continue this process until accuracy on the validation set does not increase further.

Overfitting is not just a problem for Decision Trees; it applies to regression and all other ML algorithms.

- 1 **Complexity of the hypothesis:** More complex hypotheses are more flexible but are highly prone to overfitting.
- 2 **Size of the training set:** If the training size is too small, complex models will overfit. As N (data size) increases, error generally decreases.
- 3 **Generalization error:** Our ultimate goal is minimizing error on true, unseen populations, not just memorizing the training data.

37 / 37